JC HIDRÁULICA

PEÇAS, MANUTENÇÃO E SERVIÇOS NAVAIS

EQUIPE DE DESENVOLVIMENTO

Integrantes com foco no desenvolvimento completo da aplicação, do backend ao frontend.

GRAZIELLI FONSECA

FullStack Developer

KASSYANE YUMI

FullStack Developer

COMO FUNCIONA O PROJETO

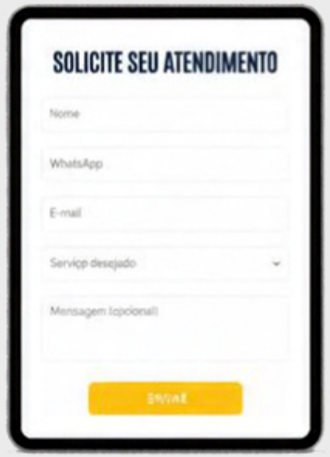
O site da JC Hidráulica funciona como uma vitrine digital
que conecta clientes ao atendimento humano de forma rápida e eficiente.

01

NAVEGAÇÃO / VITRINE

O cliente explora o catálogo
e conhece nossas soluções.

02



TRIAGEM VIA SITE

O site coleta os dados do cliente
e o serviço desejado.

03

FECHAMENTO HUMANO

Nosso vendedor entra em contato
e finaliza o atendimento via WhatsApp.

BACKEND:

ARQUITETURA E REGRAS DE NEGÓCIO

TECNOLOGIAS UTILIZADAS

- JVM/OpenJDK
- Spring Boot
- Java
- H2 Database
- Spring Data JPA

REQUISITOS DO SISTEMA

Requisitos funcionais

- Exibição e CRUD de produtos
- Status
- Envio dados triagem

Requisitos não funcionais

- Segurança (login com bcrypt, proteção API)
- Desempenho
- Preparação para integração futura

MODEL

O coração dos dados e da estrutura.

Define o formato das informações que circulam pelo sistema.

```java
@Entity // Define a classe como uma entidade JPA (tabela no banco)
@Table(name = "tb_produto") // Nome da tabela no banco de dados
@Getter
@Setter
@NoArgsConstructor
public abstract class Produto {

    @Id // Define o atributo como chave primária
    @GeneratedValue(strategy = GenerationType.IDENTITY) // Auto-incremento do ID pelo banco
    private Long id;

    @Column(nullable = false) // Torna a coluna obrigatória
    private String nome; // Armazena o nome do produto

    @Column(nullable = false)
    private String descricao;

    @Column(nullable = false)
    private String URL;

    @Column(nullable = false)
    private Boolean prudutoDisponivel;
}
```

```java
@Entity
@Table ( name ="tb_produto")
@Getter
@Setter
@NoArgsConstructor

public abstract class Produto{

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column (nullable = false)
    private String nome;

    @Column (nullable = false)
    private String descricao;

    @Column (nullable = false)
    private String URL;

    @Column (nullable = false)
    private Boolean prudutoDisponivel;

}
```

REPOSITORY

A ponte com a persistência.

Camada responsável pela comunicação direta com o banco de dados.

```java
public interface AdminRepository extends JpaRepository<Admin, Long> { // Define uma interface de repositório Spring Data JPA para a entidade Admin

    boolean existsByDocumento(String documento); // Método de consulta automática no banco

    Optional<Admin> findByEmail(String email); // Busca um admin pelo e-mail
}
```

```java
public interface ProdutoRepository extends JpaRepository<Produto, Long> {

    boolean existsByNome(String nome);

    Optional<Produto> findByNome(String nome);

    List<Produto> findByProdutoDisponivel(Boolean produtoDisponivel);
}
```

SERVICE

Onde a lógica acontece.

Processa os dados recebidos antes de salvá-los ou enviá-los de volta.

```java
@Service // Define a classe como um componente de serviço do Spring (contém a regra de negócio)
public class AdminService {

    private final AdminRepository adminRepository; // Repositório para acessar os dados de
administradores
    private final ProdutoRepository produtoRepository; // Repositório para acessar os dados de produtos

    public AdminService(AdminRepository adminRepository, ProdutoRepository produtoRepository) {
        this.adminRepository = adminRepository;
        this.produtoRepository = produtoRepository;
    }

    public Admin login(String email, String senha) {
        Admin admin = adminRepository.findByEmail(email)
                .filter(a -> a.getSenha().equals(senha))
                .orElseThrow(() -> erro("E-mail ou senha incorretos."));
        return admin;
    }

    public List<Produto> listarProdutos() {
        return produtoRepository.findAll();
    }

    private ResponseStatusException erro(String mensagem) {
        return new ResponseStatusException(HttpStatus.BAD_REQUEST, mensagem);
    }
}
```

```java
    public Produto cadastrar(Produto produto) {
        validarNomeUnico(produto.getNome());
        if (produto.getPrudutoDisponivel() == null) {
            produto.setPrudutoDisponivel(true);
        }
        return produtoRepository.save(produto);
    }

    public Produto editar(Long id, Produto produtoAtualizado) {
        Produto produtoExistente = buscarPorId(id);

        produtoExistente.setNome(produtoAtualizado.getNome());
        produtoExistente.setDescricao(produtoAtualizado.getDescricao());
        produtoExistente.setURL(produtoAtualizado.getURL());
        produtoExistente.setPrudutoDisponivel(produtoAtualizado.getPrudutoDisponivel());

        return produtoRepository.save(produtoExistente);
    }

    public Produto alternarStatusDisponibilidade(Long id) {
        Produto produto = buscarPorId(id);

        boolean novoStatus = !produto.getPrudutoDisponivel();
        produto.setPrudutoDisponivel(novoStatus);

        return produtoRepository.save(produto);
    }

    public void excluir(Long id) {
        Produto produto = buscarPorId(id);
        produtoRepository.delete(produto);
    }

    public List<Produto> listar() {
        return produtoRepository.findAll();
    }

    public Produto buscarPorId(Long id) {
        return produtoRepository.findById(id)
                .orElseThrow(() -> new ResponseStatusException(HttpStatus.NOT_FOUND,
                        "Produto " + id + " não encontrado."));
    }

    private void validarNomeUnico(String nome) {
        if (produtoRepository.existsByNome(nome)) {
            throw erro("Já existe um produto cadastrado com o nome " + nome + ".");
        }
    }
```

CONTROLLER

A porta de entrada.

Gerencia as requisições de entrada, rotas e respostas da API.

```java
@RestController // Define a classe como um controlador REST
@RequestMapping("/api/admins") // Define a URL base para todos os endpoints deste controlador
public class AdminController {

    private final AdminService adminService; // Injeta o serviço de administradores para lidar com a lógica de negócio

    public AdminController(AdminService adminService) {
        this.adminService = adminService;
    } // Construtor usado pelo Spring para injetar a dependência do AdminService

    @PostMapping("/login") // Mapeia uma requisição HTTP POST para o endpoint "/api/admins/login"
    public Admin login(@RequestParam String email,
                       @RequestParam String senha) {
        return adminService.login(email, senha);
    }

    @GetMapping("/produtos") // Mapeia uma requisição HTTP GET para o endpoint "/api/admins/produtos"
    public List<Produto> listarProdutos() {
        return adminService.listarProdutos();
    }
}
```

```java
    @PostMapping
    public Produto cadastrar(@RequestBody Produto produto) {
        return produtoService.cadastrar(produto);
    }

    @GetMapping
    public List<Produto> listar() {
        return produtoService.listar();
    }

    @GetMapping("/{id}") // Mapeia uma requisição HTTP GET passando o ID na URL
    public Produto buscar(@PathVariable Long id) {
        return produtoService.buscarPorId(id);
    }

    @PutMapping("/{id}") // Mapeia uma requisição HTTP PUT para atualizar
    public Produto editar(@PathVariable Long id, @RequestBody Produto produto) {
        return produtoService.editar(id, produto);
    }

    @PatchMapping("/{id}/status") // Mapeia uma requisição HTTP PATCH para alterar parcialmente um recurso (status de disponibilidade)
    public Produto alternarStatus(@PathVariable Long id) {
        return produtoService.alternarStatusDisponibilidade(id);
    }

    @DeleteMapping("/{id}") // Mapeia uma requisição HTTP DELETE para remover um registro pelo ID
    public void excluir(@PathVariable Long id) {
        produtoService.excluir(id);
    }
}
```

FRONTEND:
INTERFACE E TECNOLOGIAS

Uma interface robusta e ágil, com navegação clara e experiência fluida do início ao fim.

LINK DO SITE

TECNOLOGIAS UTILIZADAS

- React
- Vite
- Tailwind CSS
- JavaScript
- HTML

PERGUNTAS?

Este é o momento para esclarecer suas dúvidas sobre o projeto.

Obrigada!

Agradecemos a atenção!